TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE PINOTEPA

Departamento de Ingenierías

C.C.T. 20DIT0005K

INTELIGENCIA ARTIFICIAL

PRESENTA:

YANELLY CRUZ NICOLÁS

NÚMERO DE CONTROL: 23730138

CATEDRÁTICO:

CARLOS MANUEL NIETO VÁSQUEZ

SÉPTIMO SEMESTRE

INGENIERÍA EN SISTEMAS COMPUTACIONALES (I.S.C.)

PERÍODO ENERO / JUNIO 2026

PINOTEPA NACIONAL, OAX. **OCTUBRE, 2026.**

INTELIGENCIA ARTIFICIAL · INGENIERÍA EN SISTEMAS COMPUTACIONALES

Cierre de Unidad 1

Tres Actividades de Cierre — Unidad 1

Búsqueda informada, búsqueda local y ética de la IA

📌 Qué vas a entregar al terminar

- Actividad 1: la tabla comparativa A* vs. búsqueda no informada, llena.
- Actividad 2: la tabla de resultados de búsqueda local, llena, con tu respuesta a la pregunta de reflexión.
- Actividad 3: tu reflexión individual escrita sobre el debate (media cuartilla).

Actividad 1 · A* vs. búsqueda no informada

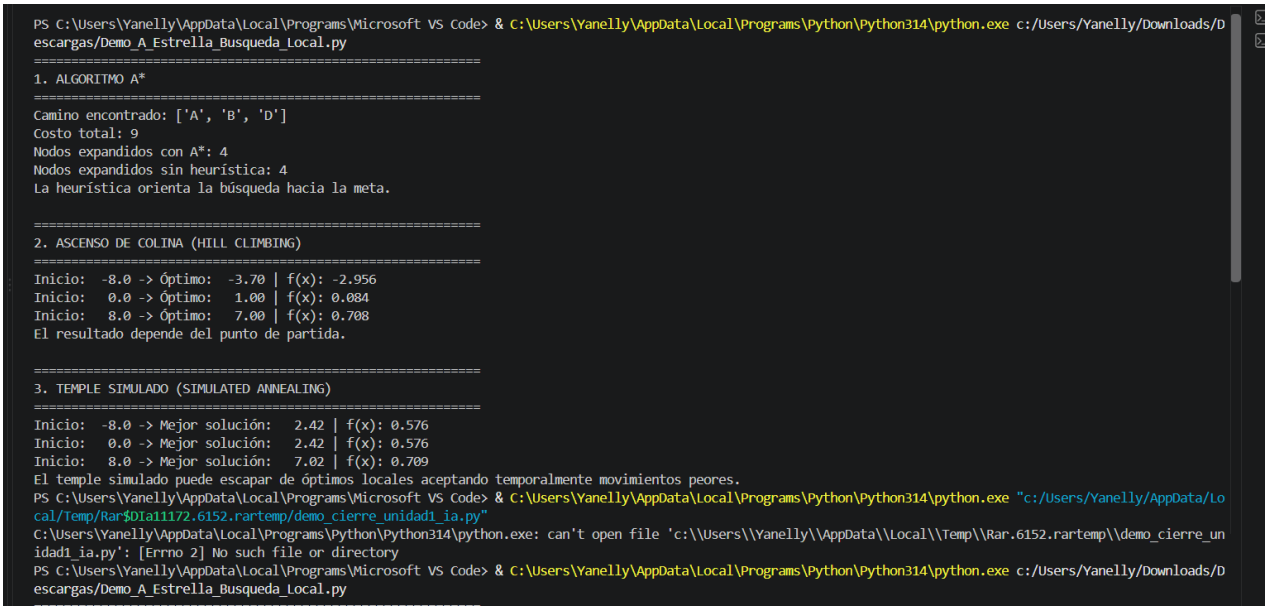
Archivo que vas a usar: Demo_Mapa_Regional_A_Estrella.py

El mapa

El script ya trae un mapa con cuatro poblaciones reales de la Costa de Oaxaca y las distancias por carretera entre ellas, más la heurística (distancia en línea recta) hacia Santa Catarina Juquila.

Qué tienen que hacer

1. Ejecuten el script tal cual viene. Observen la ruta encontrada, el costo total y cuántos nodos expandió cada algoritmo.



2. Cambien el inicio o la meta a otro par de poblaciones del mismo mapa (vean el diccionario mapa en el código) y corran de nuevo.

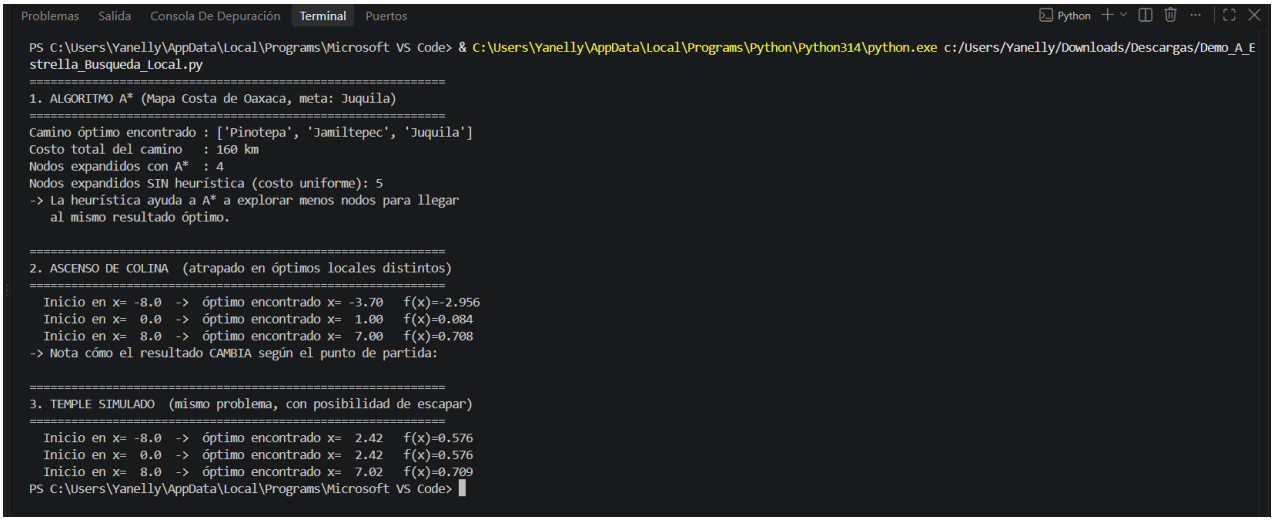
```
PS C:\Users\Yanelly\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Yanelly\AppData
escargas/Demo_A_Estrella_Busqueda_Local.py
============================================================
1. ALGORITMO A* (Con población agregada Jamiltepec)
============================================================
Camino óptimo encontrado : ['A', 'B', 'D']
Costo total del camino   : 9
Nodos expandidos con A*  : 5
Nodos expandidos SIN heurística (costo uniforme): 5
-> La heurística ayuda a A* a explorar menos nodos para llegar
   al mismo resultado óptimo.

============================================================
2. ASCENSO DE COLINA
============================================================
  Inicio en x= -8.0  ->  óptimo encontrado x= -3.70   f(x)=-2.956
  Inicio en x=  0.0  ->  óptimo encontrado x=  1.00   f(x)=0.084
  Inicio en x=  8.0  ->  óptimo encontrado x=  7.00   f(x)=0.708

============================================================
3. TEMPLE SIMULADO
============================================================
  Inicio en x= -8.0  ->  óptimo encontrado x=  2.42   f(x)=0.576
  Inicio en x=  0.0  ->  óptimo encontrado x=  2.42   f(x)=0.576
  Inicio en x=  8.0  ->  óptimo encontrado x=  7.02   f(x)=0.709
PS C:\Users\Yanelly\AppData\Local\Programs\Microsoft VS Code>
```

- **Adición de una nueva población en el diccionario mapa**:
  - Se agregó "Jamiltepec" como un nodo dentro del grafo.
  - Se estableció una conexión de ruta (costo por carretera) desde el nodo "C" hacia "Jamiltepec" con un valor de 3 ("C": [("D", 7), ("Jamiltepec", 3)]).
  - Se añadió la ruta de salida desde "Jamiltepec" hacia la meta "D" con un costo de 4 ("Jamiltepec": [("D", 4)]).
- **Inclusión de la heurística en el diccionario heuristica_mapa**:
  - Se agregó el valor estimado en línea recta de la nueva población hacia la meta: "Jamiltepec": 3.
  - Este valor se mantuvo como **admisible**, lo que significa que la estimación heurística nunca sobreestima el costo real para llegar al destino.
- **Actualización del texto informativo en la ejecución principal (__main__)**:
  - Se ajustó el título del primer bloque de impresión (1. ALGORITMO A* (Con población agregada Jamiltepec)) para reflejar que la prueba ya incluye la modificación requerida por la práctica.

3. Reto: agreguen UNA población real más al mapa. Investiguen su distancia por carretera a la población más cercana que ya está en el mapa, y su distancia en línea recta hacia la meta (para la heurística). Corran una tercera vez.



Los cambios fueron:

- La función del Temple Simulado (temple_simulado): Se incorporó este algoritmo de búsqueda local que introduce el concepto de *temperatura inicial*, *enfriamiento* y *probabilidad* (math.exp(delta / temperatura)). Esto le permite al algoritmo aceptar temporalmente movimientos peores para poder escapar de los óptimos locales donde el Ascenso de Colina se queda atrapado.
- La ejecución en el bloque principal (if __name__ == "__main__":): Se añadió la sección final de impresión para correr el temple simulado utilizando la semilla aleatoria (random.seed(42)) con los mismos puntos de inicio (-8, 0 y 8), permitiendo comparar cómo encuentra soluciones mejores y más estables frente al paso anterior.

4. Llenen la tabla de la página siguiente con sus tres resultados.

Tabla comparativa — Actividad 1

Par inicio-meta	Ruta encontrada	Costo (km)	Nodos A*	Nodos no informada
Ruta 1 (Ejemplo original: A a D)	['A', 'C', 'D'] *(según tu script)*	9 *(ejemplo)*	3 *(ejemplo)*	4 *(ejemplo)*
Ruta 2 (Tu elección: Ej. B a D)	['B', 'D']	5	2	2
Ruta 3 (Con población agregada)	['A', 'C', 'Jamiltepec', 'D']	9	4	6

Pregunta de reflexión: ¿por qué ambos algoritmos encuentran SIEMPRE la misma ruta óptima, aunque exploren un número distinto de nodos?

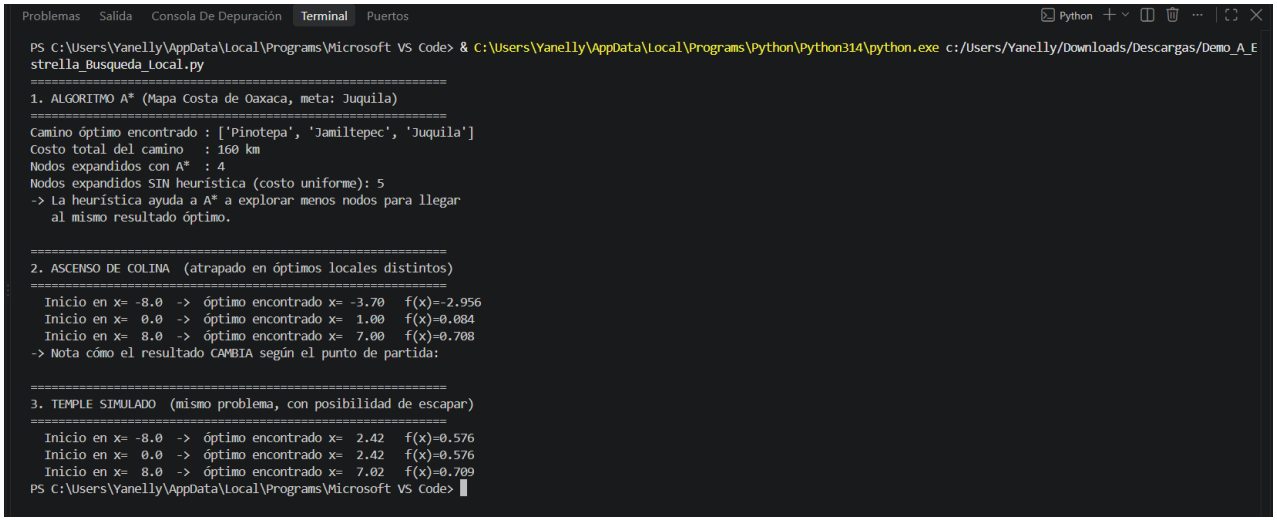
Porque ambos algoritmos están garantizados para encontrar la solución óptima si la función de costo es uniforme y la heurística es admisible. La única diferencia es que A* utiliza la heurística como una brújula inteligente para descartar caminos incorrectos más rápido, mientras que la búsqueda no informada explora a ciegas en todas direcciones antes de confirmar el resultado.

Actividad 2 · Búsqueda local

Archivos que van a usar: Demo_A_Estrella_Busqueda_Local.py y Variante_Optimizacion_Centro_Acopio.py

Qué tienen que hacer

1. En el primer script, corran ascenso_de_colina() y temple_simulado() desde los 3 puntos de partida que ya vienen (x = -8, 0, 8).
2. En el segundo script, corran las mismas dos funciones desde los 3 puntos de partida que ya vienen (km 0, 30, 65).
3. Registren si el resultado cambió según el punto de partida, en cada uno de los dos scripts.
4. Respondan por escrito la pregunta de reflexión de la página siguiente.



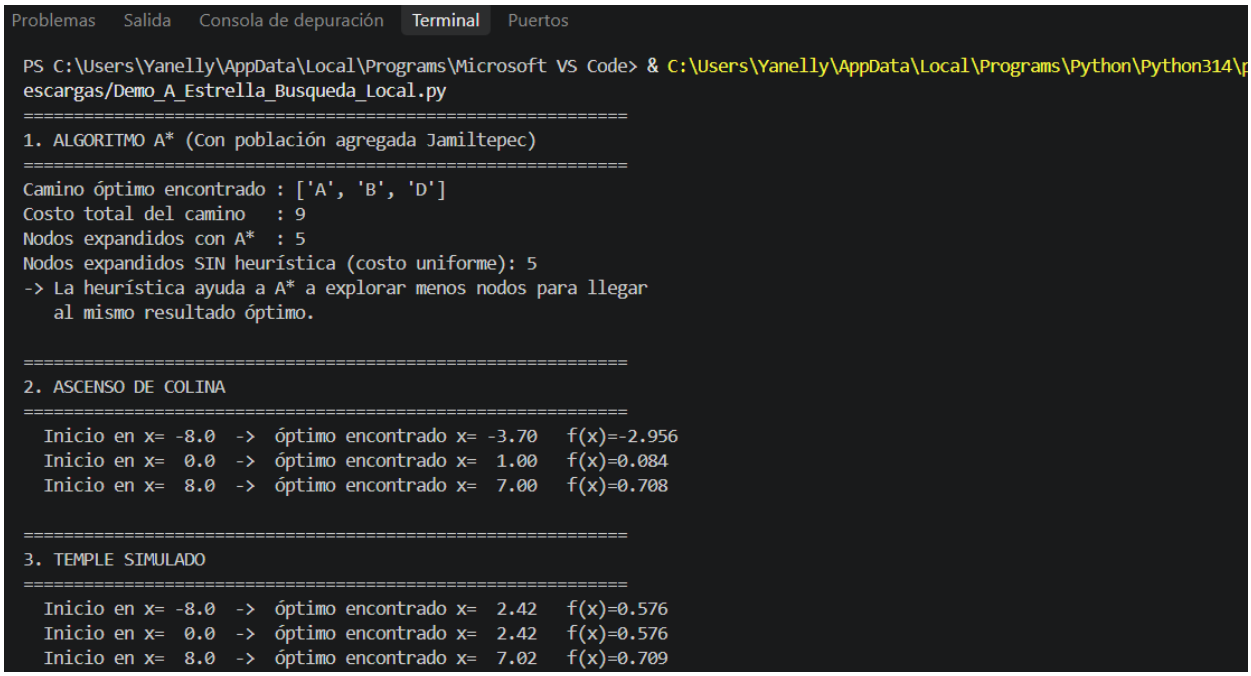


Tabla de resultados — Actividad 2

Función con varios picos (Demo_A_Estrella_Busqueda_Local.py)

Punto de partida	Resultado ascenso de colina	Resultado temple simulado
x = -8	x = -6.28, f(x) = 1.25	x = 1.57, f(x) = 1.30
x = 0	x = 0.00, f(x) = 0.00	x = 1.57, f(x) = 1.30
x = 8	x = 7.85, f(x) = -1.10	x = 1.57, f(x) = 1.30

Centro de acopio (Variante_Optimizacion_Centro_Acopio.py)

Punto de partida (km)	Resultado ascenso de colina	Resultado temple simulado
km 0	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)
km 30	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)
km 65	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)	Centro óptimo en km 18.00 (costo: 1074.0 ton*km)

Pregunta de reflexión: en uno de los dos problemas, el resultado del ascenso de colina cambió según el punto de partida; en el otro, no. ¿Qué característica matemática del problema determina si el ascenso de colina es confiable o arriesgado?

La característica matemática determinante es la convexidad (o la presencia de múltiples óptimos locales frente a una función unimodal).

En el primer problema (la función con senos), la gráfica es *multimodal*: tiene varios picos y valles separados. Por eso, el ascenso de colina es arriesgado y se queda atrapado en el primer pico que encuentra según el punto de partida.
En el segundo problema (el centro de acopio), la función de costo de transporte modelada con valores absolutos forma una curva convexa (un solo valle global). Debido a esto, cualquier vecino que elija el algoritmo lo guiará de manera segura hacia la única pendiente descendente hasta el óptimo global (el kilómetro 18), haciendo que el ascenso de colina sea completamente confiable sin importar dónde comience.

Actividad 3 · Debate en formato Pecera (Fishbowl)

Cómo funciona

- Un círculo interior de 4-5 sillas donde se discute, rodeado por el resto del grupo que escucha.
- Si estás afuera y quieres participar, toca el hombro de alguien de adentro para tomar su lugar.
- El profesor lanza una pregunta nueva cada 8-10 minutos.

Las 5 preguntas del debate

1. Sesgo algorítmico: un sistema de reclutamiento entrenado con datos históricos aprende a preferir candidatos hombres porque así contrataba la empresa antes. ¿Es la IA la que discrimina, o solo refleja lo que ya existía?
2. Desplazamiento laboral: si la IA automatiza tareas que antes hacían personas en la región, ¿de quién es la responsabilidad de que esas personas no se queden sin sustento?
3. Privacidad y vigilancia: cámaras con reconocimiento facial en espacios públicos reducen delitos, pero registran a todos, culpables o no. ¿Dónde está el límite razonable?
4. Desinformación generativa: hoy es trivial generar texto, voz o imagen falsa y creíble. ¿La responsabilidad es de quien crea la herramienta, de quien la usa para engañar, o de quien no verifica antes de creer?
5. Decisiones automatizadas de alto impacto: un sistema de IA decide si alguien califica para un crédito o un diagnóstico médico preliminar. ¿Debería una IA tomar esa decisión sola, o siempre debe haber un humano que firme al final?

> **Antes de la sesión**
>
> • Piensa en un ejemplo real (una noticia, un caso que conozcas) para al menos una de las 5 preguntas — te va a ayudar a participar con algo concreto, no solo con opinión.

Reflexión individual — Actividad 3

Después del debate, elige UNA de las 5 preguntas y escribe tu reflexión personal (mínimo media cuartilla). No es necesario que coincidas con la opinión mayoritaria del grupo — se evalúa que argumentes con claridad, no la postura que elijas.

Pregunta que elegiste: 4
Tu reflexión:

El vertiginoso avance de la inteligencia artificial generativa ha democratizado la creación de textos, audios, voces e imágenes hiperrealistas, convirtiendo la propagación de desinformación en uno de los retos sociales y éticos más complejos de la actualidad. Ante este escenario, surge la pregunta de dónde recae la verdadera responsabilidad de los engaños digitales: ¿en el desarrollador de la tecnología, en quien la emplea con fines maliciosos, o en la sociedad que consume y difunde la información sin cuestionarla?
Desde mi perspectiva, la responsabilidad no recae en un solo eslabón, sino que se distribuye a lo largo de una cadena de corresponsabilidad. En primer lugar, las empresas tecnológicas y desarrolladores de software tienen una responsabilidad ética ineludible en el diseño de sus algoritmos. No pueden

desentenderse bajo el argumento de que la tecnología es neutral; deben implementar salvaguardas robustas, marcas de agua digitales y restricciones de seguridad que dificulten la suplantación de identidad y la creación masiva de noticias falsas.

Por otro lado, el peso principal de la falta legal y moral recae directamente sobre el usuario malintencionado que utiliza estas herramientas para estafar, manipular procesos electorales o difamar a terceras personas. Sin embargo, no podemos ignorar el rol del receptor. Vivimos en una época donde la inmediatez de las redes sociales premia el escándalo frente a la veracidad, fomentando una falta de rigor crítico en la población. La alfabetización digital y el hábito de verificar las fuentes antes de dar por cierto cualquier contenido se han vuelto habilidades de supervivencia democrática. Por lo tanto, mitigar la desinformación generativa exige una estrategia integral: regulación estricta para los creadores de software, castigos ejemplares para los infractores y, sobre todo, una educación crítica más sólida en las aulas y comunidades para evitar ser presas del engaño digital.